Git: Software Engineering Lab Practice

Set 1: Repository Initialization, Stashing, and Merging

I. Initialize a new local Git repository in a directory named `Nexus-Portal`.

II. Set your username to `NexusDev` and your email to `dev@nexus.com` globally.

III. Create a new file named `global_layout.css`. Stage the file.

IV. Before you can commit, you receive an urgent request. Use Git to temporarily shelf (stash) your uncommitted changes.

V. Create a new branch named `hotfix-sata`, switch to it, add a file named `sata_hdd_fix.js`, and commit with the message "Fix SATA HDD external drive read error".

VI. Switch back to the `master` branch and merge the `hotfix-sata` branch.

VII. Retrieve and apply your temporarily stashed changes, then commit `global_layout.css`.

Set 2: Soft Resets, Reverts, and Branch Management

I. Initialize a new local Git repository named `Hospital-System` and create an initial commit with a `readme.md` file to establish the `master` branch.

II. Create two new branches named `feature-patient-reg` and `feature-doctor-select`.

III. Move to the branch `feature-patient-reg`, add a file named `self_register.html`, and commit with the message "Add ptiant interface".

IV. You realize you misspelled "patient" in the commit message. Perform a **soft reset** to go back one commit while keeping your files staged, then recommit with the correct message "Add patient interface".

V. Move to the branch `feature-doctor-select`, add a file named `select_logic.js`, and commit with the message "Add doctor selection logic".

VI. You test the code and realize `select_logic.js` breaks the routing. **Revert** the commit that added this file without rewriting the commit history.

VII. Merge both branches back into the `master` branch.

Set 3: History Inspection, Hard Resets, and Merge Conflicts

I. Initialize a new local Git repository named `CalculatorApp`. Create a file named `math_functions.txt`, add the text `Base Logic`, and make an initial commit to establish the `master` branch.

II. Create a new branch named `advanced-math` and switch to it.

III. Open `math_functions.txt`, replace the text with `Multiplication Logic`, and commit the change with the message "Add multiplication".

IV. Open the same file, replace the text with `Divide by Zero Logic`, and commit with the message "Add experimental division".

V. You realize the division logic is completely broken. Use the appropriate Git command to view your **commit log** and find the exact commit hash (code) for the "Add multiplication" commit.

VI. Perform a **hard reset** using that commit hash. This will permanently erase the experimental division commit and revert your file back to the multiplication state.

VII. Switch back to the `master` branch. Open `math_functions.txt`, replace the text with `Addition Logic`, and commit with the message "Add addition".

VIII. Merge the `advanced-math` branch into `master`. Since both branches modified the exact same line in different ways, this will trigger a **merge conflict**.

IX. Resolve the conflict in your text editor by keeping both the addition and multiplication logic, stage the resolved file, and finalize the merge.